

Documentação de Projeto — VetCare+

Sistema de Gestão para Clínica Veterinária

Versão 1.0 — 27 de maio de 2026

Trabalho desenvolvido por **Luiz F. Moreira** como parte da disciplina de Projeto de Software.

Histórico de Revisões

Versão	Data	Autor	Mudança
0.1	08/05/2026	Luiz	Esboço inicial dos casos de uso e diagrama de classes.
0.2	12/05/2026	Luiz	Atores externos definidos; enums adicionados às classes.
0.3	15/05/2026	Luiz	Primeiro diagrama de sequência (Agendar Consulta).
0.4	19/05/2026	Luiz	Regras de Negócio (RN-01 a RN-15) e contratos de operação.
0.5	22/05/2026	Luiz	Diagramas de estado; sequências adicionais; refatoração para microsserviços.
1.0	27/05/2026	Luiz	Revisão final; descrição expandida dos UCs; entrega.

Sumário

- 1. [Introdução](#)
- 2. [Modelos de Usuário e Requisitos](#)
 - 2.1 Descrição dos Atores
 - 2.2 Regras de Negócio
 - 2.3 Modelo de Casos de Uso
 - 2.4 Diagrama de Casos de Uso
- 3. [Contratos de Operação](#)
- 4. [Modelos de Projeto](#)
 - 4.1 Arquitetura
 - 4.2 Componentes e Implantação
 - 4.3 Diagrama de Classes
 - 4.4 Diagramas de Sequência
 - 4.5 Diagramas de Comunicação
 - 4.6 Diagramas de Estado
- 5. [Modelos de Dados](#)
- 6. [Escopo do MVP e Versões Futuras](#)

1. Introdução

O VetCare+ é um sistema de gestão para clínicas veterinárias. Cobre agendamento de consultas, prontuário eletrônico do pet, controle de vacinação com lembretes automáticos, pagamento integrado (Pix e cartão) e relatórios financeiros para o administrador.

A motivação veio de um problema real: minha tia toca uma clínica veterinária de bairro e, até hoje, controla agenda em caderno e ficha do animal em papel. Quando cliente liga pra remarcar, é confusão. Quando vacina vence, ninguém avisa. Quando o mês acaba, ela tem que somar recibo a mão pra saber quanto entrou.

Este documento descreve a modelagem completa do sistema — atores, regras de negócio, casos de uso, arquitetura, classes, sequências, estados e modelo de dados. O foco é arquitetural e de design; não há código implementado.

O sistema foi projetado em **microsserviços** porque, embora uma clínica única caiba num monolito, o desenho aqui foi pensado pra escalar pra uma rede de clínicas (modelo Petz/Cobasi). Os limites entre serviços seguem fronteiras de domínio: agendamento, prontuário clínico e pagamento são contextos diferentes com ciclos de mudança diferentes.

2. Modelos de Usuário e Requisitos

2.1 Descrição dos Atores

Tutor — Dono do pet. Cria sua conta com CPF, cadastra um ou mais animais e usa o sistema pra agendar consultas, consultar o histórico do pet (prontuário, vacinas), pagar atendimentos pelo app e receber lembretes de vacina vencendo. É o ator com menor privilégio: só enxerga os próprios pets.

Recepcionista — Funcionária do balcão. Faz quase tudo que o tutor faz, mas em nome dele: agenda e cancela consulta por telefone, cadastra pet quando o tutor chega sem conta, registra o pagamento presencial e visualiza a agenda do dia pra organizar a sala de espera. Não acessa o prontuário clínico em detalhe.

Veterinário — Profissional com CRMV ativo. Atende a consulta, preenche o prontuário (anamnese, diagnóstico, observações), aplica vacinas seguindo o protocolo da espécie, prescreve medicamentos e solicita exames. Só consegue editar prontuário das consultas que ele está atendendo no momento.

Administrador — Dono ou gerente da clínica. Cuida do cadastro de veterinários (admissão e desligamento), define a tabela de serviços e preços, emite relatórios financeiros mensais e gerencia permissões. É o único que pode reabrir consultas finalizadas ou estornar pagamentos. Em clínica pequena, normalmente é a mesma pessoa que toca o balcão.

Atores externos:

- **Gateway de Pagamento** — serviço externo (Pix e cartão) que processa transações e devolve aprovação por webhook.
- **Serviço de E-mail (SMTP)** — gateway externo (ex.: SendGrid) que entrega lembretes e recibos.

2.2 Regras de Negócio

ID	Regra
RN-01	Um veterinário não pode ter duas consultas no mesmo horário. Janela mínima entre consultas: 30 minutos.

ID	Regra
RN-02	Cancelamento com menos de 2 horas de antecedência gera taxa de 50% do valor da consulta.
RN-03	Pet deve estar vinculado a exatamente 1 tutor responsável, com CPF cadastrado.
RN-04	Vacina antirrábica tem reforço anual obrigatório. Sistema dispara lembrete 30 dias e 7 dias antes do vencimento.
RN-05	Vacinas V8/V10 seguem protocolo: 1ª dose após 45 dias de vida, reforço a cada 21 dias até 4 meses, depois anual.
RN-06	Prescrição de medicamento controlado exige CRMV ativo do veterinário no momento da prescrição.
RN-07	Consulta só passa para REALIZADA depois do prontuário preenchido (anamnese e diagnóstico no mínimo).
RN-08	Pagamento via Pix tem janela de 15 minutos para confirmação. Após isso, a consulta volta para AGUARDANDO_PAGAMENTO.
RN-09	Tutor menor de 18 anos precisa de responsável legal cadastrado (CPF e grau de parentesco).
RN-10	Veterinário só pode atender espécies da sua especialidade declarada, salvo override do administrador.
RN-11	Relatório financeiro mensal consolida pagamentos com status CONFIRMADO entre o 1º e o último dia do mês.
RN-12	E-mail de lembrete tenta envio até 3 vezes em caso de falha. Depois disso, é descartado e admin é notificado.
RN-13	Histórico do prontuário é imutável. Correções entram como nova anotação com referência à anterior.
RN-14	Senha mínima de 8 caracteres com hash BCrypt. Para perfis internos (recepcionista, vet, admin) expira em 90 dias.
RN-15	Pet inativado (óbito ou transferência) não pode ser agendado, mas o histórico permanece consultável.

2.3 Modelo de Casos de Uso

O sistema tem 16 casos de uso. Os UCs marcados como "include" são incluídos por todos os outros que exigem usuário autenticado, evitando repetição na modelagem.

UC-01 — Autenticar. Incluído por todos os UCs autenticados. Verifica e-mail e senha (BCrypt), aplica RN-14 e emite JWT válido por 24 horas. Ator: Usuário genérico.

UC-02 — Gerenciar Perfil. Permite que qualquer usuário autenticado atualize seus dados cadastrais e troque a senha. Ator: Usuário.

UC-03 — Cadastrar Pet. Tutor cadastra um novo pet vinculado ao próprio CPF (RN-03). Validações de espécie, peso e data de nascimento. Ator principal: Tutor.

UC-04 — Agendar Consulta. Tutor (ou recepcionista em nome dele) reserva um horário com um veterinário. Inclui UC-01 e UC-16. Pré-condições: pet ativo (RN-15), vet com agenda aberta, especialidade compatível (RN-10), sem conflito de horário (RN-01).

UC-05 — Cancelar Consulta. Tutor, recepcionista ou admin cancela uma consulta agendada. Se for com menos de 2h de antecedência, aplica taxa de 50% (RN-02). Se já estiver paga, dispara estorno via ms-pagamento.

UC-06 — Reagendar Consulta. Variação do cancelamento que mantém o ID da consulta original e move pra nova data. Útil pra preservar o histórico vinculado.

UC-07 — Atender Consulta. Veterinário inicia o atendimento e preenche o prontuário. Inclui UC-08 (registrar prontuário). Pode estender pra UC-09 (prescrição) e UC-10 (vacina). Ao concluir, marca consulta como REALIZADA (RN-07).

UC-08 — Registrar Prontuário. Vet preenche anamnese, diagnóstico e observações. Imutável após gravado (RN-13). Vincula automaticamente à consulta em andamento.

UC-09 — Prescrever Medicamento. Vet adiciona prescrição ao prontuário com posologia e duração. Se medicamento for controlado, valida CRMV ativo (RN-06).

UC-10 — Aplicar Vacina. Vet registra aplicação de vacina, lote e calcula próxima dose automaticamente conforme protocolo (RN-04, RN-05). Agenda lembretes 30 e 7 dias antes do vencimento.

UC-11 — Pagar Consulta. Tutor (ou recepcionista) inicia pagamento via Pix ou cartão. Integra com Gateway de Pagamento externo. Pix expira em 15 minutos sem confirmação (RN-08).

UC-12 — Visualizar Agenda do Dia. Veterinário e recepcionista veem a lista de consultas do dia, com status (agendada, confirmada, em atendimento, realizada).

UC-13 — Ver Prontuário do Pet. Tutor consulta histórico do próprio pet. Veterinário consulta histórico de qualquer pet da clínica.

UC-14 — Gerenciar Veterinários. Administrador cadastra, ativa ou desliga veterinários. Define especialidade e carga horária.

UC-15 — Gerar Relatório Financeiro. Administrador gera relatório mensal consolidando pagamentos confirmados (RN-11). Exporta em PDF.

UC-16 — Notificar. Incluído por UC-04, UC-05, UC-10. Encapsula a lógica de envio de notificações (e-mail) com retry e descarte após 3 falhas (RN-12).

2.4 Diagrama de Casos de Uso

Fonte: [Codigo/CasosDeUso/diagrama-de-caso-de-uso.puml](#) Imagem: [Modelagem/CasosDeUso/diagrama-de-caso-de-uso.png](#)

O diagrama mostra a hierarquia de atores (Usuário abstrato → Tutor, Recepcionista, Veterinário, Administrador), os 16 casos de uso e suas relações `<<include>>` e `<<extend>>`.

3. Contratos de Operação

Cada contrato descreve uma operação principal do sistema, com pré e pós-condições.

CT-01 — agendarConsulta

- **Operação:** `agendarConsulta(tutorId: Long, petId: Long, vetId: Long, dataHora: DateTime, motivo: String): ConsultaDTO`
- **UC:** UC-04
- **Pré-condições:** tutor autenticado; pet pertence ao tutor e está ATIVO; vet está ATIVO; dataHora futura; especialidade compatível com espécie (RN-10).
- **Pós-condições:** consulta persistida com status AGENDADA; evento `ConsultaAgendadaEvent` publicado no RabbitMQ; lembrete agendado pra 24h antes.

CT-02 — cancelarConsulta

- **Operação:** `cancelarConsulta(consultaId: Long, motivo: String, solicitanteId: Long): ResultadoCancelamento`
- **UC:** UC-05
- **Pré-condições:** consulta existe e está em AGENDADA ou CONFIRMADA; solicitante é o tutor dono, recepcionista ou admin; dataHora futura.
- **Pós-condições:** status atualizado para CANCELADA; se antecedência < 2h, taxa de 50% aplicada (RN-02); se consulta paga, estorno disparado; tutor notificado.

CT-03 — registrarProntuario

- **Operação:** `registrarProntuario(consultaId: Long, vetId: Long, anamnese: String, diagnostico: String, obs: String): ProntuarioDTO`
- **UC:** UC-08
- **Pré-condições:** consulta em EM_ATENDIMENTO; vetId é o atendente; anamnese e diagnóstico não vazios.
- **Pós-condições:** prontuário criado e imutável (RN-13); consulta passa pra REALIZADA; cobrança gerada.

CT-04 — prescreverMedicamento

- **Operação:** `prescreverMedicamento(prontuarioId: Long, vetId: Long, medicamentoId: Long, posologia: String, duracaoDias: int): PrescricaoDTO`
- **UC:** UC-09
- **Pré-condições:** prontuário existe; vet com CRMV ativo (RN-06); medicamento cadastrado; duração > 0.
- **Pós-condições:** prescrição vinculada ao prontuário; se controlado, registro em log de auditoria.

CT-05 — aplicarVacina

- **Operação:** `aplicarVacina(prontuarioId: Long, vetId: Long, vacinaTipo: TipoVacina, lote: String): AplicacaoVacinaDTO`
- **UC:** UC-10
- **Pré-condições:** prontuário existe; vacina compatível com espécie e idade do pet; lote informado.

- **Pós-condições:** aplicação registrada; `proximaDose` calculada por protocolo (RN-04, RN-05); lembrete de reforço agendado.

CT-06 — pagarConsulta

- **Operação:** `pagarConsulta(consultaId: Long, metodo: MetodoPagamento, valor: BigDecimal): ResultadoPagamento`
- **UC:** UC-11
- **Pré-condições:** consulta REALIZADA; pagamento ainda não confirmado; valor confere com o valor da consulta.
- **Pós-condições:** cobrança gerada no gateway; se Pix expirar em 15 min (RN-08), status volta pra EXPIRADO; recibo enviado por e-mail em caso de aprovação.

CT-07 — cadastrarPet

- **Operação:** `cadastrarPet(tutorId: Long, dados: PetData): PetDTO`
- **UC:** UC-03
- **Pré-condições:** tutor autenticado e maior de 18 (RN-09); CPF válido; espécie em lista controlada; data de nascimento não futura.
- **Pós-condições:** pet criado com status ATIVO.

CT-08 — autenticar

- **Operação:** `autenticar(email: String, senha: String): TokenJWT`
- **UC:** UC-01
- **Pré-condições:** e-mail cadastrado; conta não BLOQUEADA; senha não expirada (RN-14).
- **Pós-condições:** hash BCrypt conferido; JWT emitido (exp 24h); login registrado em auditoria. Após 5 falhas seguidas, conta bloqueia por 15 min.

CT-09 — gerarRelatorioFinanceiro

- **Operação:** `gerarRelatorioFinanceiro(adminId: Long, mesRef: YearMonth, filtros: Map): RelatorioMensalDTO`
- **UC:** UC-15
- **Pré-condições:** solicitante é Administrador; mês de referência $\leq$ mês atual.
- **Pós-condições:** dados agregados de pagamentos CONFIRMADO no período (RN-11); PDF gerado; total bruto, por método e por veterinário disponíveis.

CT-10 — enviarLembrete

- **Operação:** `enviarLembrete(notificacaoId: Long): ResultadoEnvio`
- **UC:** UC-16
- **Pré-condições:** notificação com status PENDENTE; canal definido; destinatário com e-mail válido.
- **Pós-condições:** e-mail enviado via SMTP; status passa pra ENVIADA ou FALHA; após 3 tentativas (RN-12), notificação marcada como DESCARTADA e admin alertado.

4. Modelos de Projeto

4.1 Arquitetura

O sistema segue arquitetura de **microsserviços** com seis serviços, cada um com seu próprio banco PostgreSQL. A comunicação entre serviços é majoritariamente assíncrona via RabbitMQ — eventos de domínio são publicados quando algo relevante acontece (consulta agendada, vacina aplicada, pagamento confirmado) e consumidos por quem precisar reagir.

Serviço	Responsabilidade
ms-usuarios	Autenticação JWT, CRUD de perfis (tutor/recepcionista/vet/admin).
ms-agendamento	Consultas, agenda dos vets, regras de conflito de horário.
ms-prontuario	Anamnese, diagnóstico, vacinas, prescrições. Upload de exames pra S3.
ms-pagamento	Integração com gateway Pix/cartão, webhooks, estorno.
ms-notificacao	Worker que consome a fila e dispara e-mail. Scheduler de lembretes.
ms-relatorios	Leitura cruzada de pagamento e agendamento para relatórios.

Por dentro de cada serviço, mantive a estrutura Spring Boot em camadas:

- **Controller** recebe a requisição HTTP, valida e devolve o DTO.
- **Service** concentra a regra de negócio (ex.: "não dá pra agendar dois pets no mesmo horário pro mesmo vet").
- **Repository** conversa com o banco via Spring Data JPA.
- **DTO** isola a entidade do contrato da API.

Na frente fica um **API Gateway** que valida o JWT na borda e roteia pros serviços via Application Load Balancer. Front-end React (SPA) e React Native (mobile) entram por CloudFront → API Gateway.

Padrões de projeto adotados: Repository (acesso a dados), Service Layer (regras de negócio), DTO (transferência), Event-Driven (RabbitMQ desacopla os serviços), Strategy (cálculo de próxima dose de vacina por protocolo), Factory (criação de notificações por canal).

Decisões importantes:

1. **Microsserviços com banco por serviço.** Cada ms tem seu próprio banco, sem joins entre eles. Quando ms-relatorios precisa cruzar dados de pagamento e agendamento, lê dos dois bancos (read-only) ou consome eventos pra construir uma view materializada.
2. **Comunicação assíncrona como default.** Notificação e atualização de relatório vão por fila. REST síncrono só nos fluxos onde a resposta é parte da UX (agendar, pagar, atender).
3. **JWT stateless.** Sem sessão no servidor. Token validado no API Gateway e propagado.
4. **Banco relacional (PostgreSQL).** Domínio tem transações ACID (pagamento, agendamento), então NoSQL não compensa.

Trade-offs assumidos:

- Microsserviços trazem complexidade operacional (orquestração, observabilidade, deploy). Pra uma clínica única isso é exagero. A escolha é justificada pelo desenho voltado pra rede de clínicas.

- Consistência eventual entre serviços via fila — relatórios podem demorar alguns segundos pra refletir um pagamento recém-confirmado.

4.2 Componentes e Implantação

Fonte: [Codigo/Componentes/diagrama-de-componentes-e-implantacao.puml](#)

O diagrama mostra a separação física: clientes (Web e Mobile) → CloudFront → API Gateway → ALB → cluster Docker/ECS com os seis microsserviços e seus bancos. RabbitMQ orquestra a comunicação assíncrona. Externos: gateway de pagamento, SMTP e S3 (anexos de exame).

4.3 Diagrama de Classes

Fonte: [Codigo/Classes/diagrama-de-classes.puml](#)

O modelo de domínio gira em torno da hierarquia **Usuario** (abstrata) → Tutor, Recepcionista, Veterinario, Administrador. A interface **Autenticavel** define o contrato pra qualquer entidade autenticável.

Entidades principais: **Pet**, **Consulta** (com enum **StatusConsulta** cobrindo todo o ciclo de vida), **Prontuario**, **Vacina** e a classe associativa **AplicacaoVacina** (entre Pet, Vacina e Prontuário, carregando lote e próxima dose), **Prescricao**, **Medicamento**, **Pagamento**, **Notificacao**.

Enums modelam valores fixos: **StatusConsulta**, **TipoConsulta**, **EspecieAnimal**, **Especialidade**, **TipoVacina**, **MetodoPagamento**, **StatusPagamento**, **CanalNotificacao**, **StatusNotificacao**, **StatusPet**, **StatusUsuario**.

4.4 Diagramas de Sequência

Sete diagramas de sequência cobrem os fluxos principais. Cada um mostra interação entre cliente, API Gateway, microsserviço responsável, repositórios e (quando aplicável) RabbitMQ e serviços externos. Os fluxos alternativos (alt/opt) representam regras de negócio e exceções.

UC	Cenário	Arquivo
UC-04	Agendar Consulta	UC-04-agendar-consulta.puml
UC-05	Cancelar Consulta	UC-05-cancelar-consulta.puml
UC-07	Atender Consulta e Prontuário	UC-07-atender-consulta.puml
UC-10	Aplicar Vacina com Reforço	UC-10-aplicar-vacina.puml
UC-11	Pagar Consulta via Pix	UC-11-pagar-consulta.puml
UC-15	Gerar Relatório Financeiro	UC-15-gerar-relatorio.puml
UC-16	Job de Envio de Lembrete	UC-16-enviar-lembrete.puml

4.5 Diagramas de Comunicação

Dois diagramas de comunicação, equivalentes em informação às sequências correspondentes mas com ênfase nos relacionamentos estruturais entre objetos:

- [Comunicação UC-04 — Agendar Consulta](#)

- [Comunicação UC-10 — Aplicar Vacina](#)

4.6 Diagramas de Estado

Quatro entidades têm ciclo de vida complexo o suficiente pra justificar diagrama de estados:

- **Consulta** — do agendamento até o pagamento, passando por confirmação, atendimento, realização. Inclui transições de cancelamento e falta. [diagrama-de-estado-consulta.puml](#)
- **Vacinação** — protocolo cíclico: pendente → aplicada → vencendo → vencida → reforço aplicado. Com guards de RN-04 e RN-05. [diagrama-de-estado-vacinacao.puml](#)
- **Pagamento** — pendente → processando → aprovado/recusado/expirado → confirmado/estornado. Com timeout Pix da RN-08. [diagrama-de-estado-pagamento.puml](#)
- **Pet** — ativo → inativo transferido / óbito. Preserva histórico mesmo após inativação (RN-15). [diagrama-de-estado-pet.puml](#)

5. Modelos de Dados

5.1 Tecnologia de Persistência

Cada microserviço tem seu próprio banco **PostgreSQL 16**, sem compartilhamento direto. Acesso via Spring Data JPA + Hibernate.

A herança da hierarquia **Usuario** é tratada por **tabelas separadas compartilhando chave primária** (table-per-subclass): tabela **usuarios** é a base; **tutores**, **recepcionistas**, **veterinarios** e **administradores** estendem com **id** referenciando **usuarios.id**. Permite consultas polimórficas via JOIN e mantém integridade.

Valores enumerados (status, tipos, métodos) são armazenados como **varchar** pra facilitar leitura em SQL e relatórios, ao invés de inteiros.

5.2 Diagrama Entidade-Relacionamento

Fonte: [Codigo/ER/diagrama-er-vetcareplus.puml](#)

O modelo conta com 14 tabelas distribuídas entre os bancos dos microserviços:

Microserviço	Tabelas
ms-usuarios	usuarios , tutores , recepcionistas , veterinarios , administradores
ms-agendamento	consultas
ms-prontuario	prontuarios , vacinas , aplicacoes_vacina , medicamentos , prescricoes , pets
ms-pagamento	pagamentos
ms-notificacao	notificacoes

5.3 Índices Importantes

Tabela	Índice	Justificativa
usuarios	<code>email</code> (unique)	Login
veterinarios	<code>crm</code> (unique)	Identificação profissional
consultas	<code>(veterinario_id, data_hora)</code>	Checagem de conflito (RN-01)
consultas	<code>status</code>	Filtro de agenda do dia
pagamentos	<code>(status, pago_em)</code>	Relatório financeiro (RN-11)
aplicacoes_vacina	<code>proxima_dose</code>	Job de lembrete (RN-04)
notificacoes	<code>(status, data_envio)</code>	Job de envio (RN-12)

6. Escopo do MVP e Versões Futuras

MVP (v1.0 — entrega)

- Autenticação JWT com 4 perfis.
- CRUD de tutor e pet.
- Agendamento, cancelamento, reagendamento e atendimento de consulta.
- Prontuário com anamnese, diagnóstico, vacinas e prescrições.
- Pagamento por Pix e cartão via gateway externo.
- Lembrete por e-mail (consulta 24h antes; vacina 30 e 7 dias antes do vencimento).
- Relatório financeiro mensal.

v1.1 — Próximo ciclo

- Upload de exames complementares (PDF, imagem) pra S3.
- App mobile React Native para o tutor (visualizar agenda, prontuário, pagamento).
- Notificação push.

v2.0 — Visão de longo prazo

- Notificação por WhatsApp como canal alternativo (RN-12 expandida).
- Telemedicina veterinária (videoconsulta integrada).
- Integração com farmácia veterinária parceira (entrega de medicação).
- Programa de fidelidade.

Fora de escopo (decisão consciente)

- **Estoque de medicamentos.** Clínica pequena terceiriza a venda — não vale o esforço de modelar.
- **Internação e centro cirúrgico.** Domínio diferente, com regras próprias (anestesia, escala 24h). Mereceria um módulo separado.
- **Faturamento por convênio pet.** Mercado ainda imaturo no perfil de cliente-alvo.

Fim do documento.